

Mobile Application Development with Flutter

Lecture 60. Flutter App Architecture

What is App Architecture?

- App architecture means:
 - 👉 How we organize our app code and files
- It defines:
 - Where UI goes
 - Where logic goes
 - Where data goes
 - How files communicate

Prepared by Ehtisham Rasheed

Why Architecture is Important?

Without Architecture	With Architecture
✗ Difficult to manage	✓ Clean code
✗ Hard to debug	✓ Easy debugging
✗ Code duplication	✓ Reusable code
✗ Difficult teamwork	✓ Better teamwork
✗ Hard to scale app	✓ Scalable apps
	✓ Easy maintenance

Beginner Problem Example

- Many beginners write everything inside “main.dart”
- Inside same file:
 - UI, APIs, Logic, State management etc
 - Everything mixed together
- Goal of Architecture
- Architecture tries to separate:
 - UI, Logic, Data
 - This separation makes apps cleaner

Common Architecture Types

- Today we will briefly understand:

1. MVC
2. MVP
3. MVVM
4. Clean Architecture

Prepared by Ehtisham Rasheed

1. MVC Architecture

- MVC = Model, View, Controller
- Structure: View ↔ Controller ↔ Model
- Model handles data (UserModel, ProductModel etc)
- View contains UI Part (HomeScreen, LoginScreen etc)
- Controller handles logic (API Calls, User actions, App behavior etc)
- Problem with MVC Architecture
 - As apps grow, Controllers become very large
 - This is called Massive Controller Problem

2. MVP Architecture

- MVP = Model, View, Presenter
- Structure: View $\leftrightarrow$ Presenter $\leftrightarrow$ Model
- Model and View are same as MVC
- Presenter handles:
 - Logic, communication, data preparation
- MVP improved some MVC problems

3. MVVM Architecture

- MVVM = Model, View, ViewModel
- Structure: View $\leftrightarrow$ ViewModel $\leftrightarrow$ Model
- Model = Data (userModel, studentModel etc)
- View = UI (LoginScreen, HomeScreen etc)
- ViewModel = Logic (AuthViewModel, UserViewModel etc)
- MVVM is Popular now a day and we'll cover this in our course

4. Clean Architecture

- Very professional architecture
- Popular in enterprise apps
- Basic layers
 - Presentation (UI and state management)
 - Domain (Business rules and app logic)
 - Data (APIs, database, repositories)
- It is more advance and we'll not cover in this course

Popular Architecture

- Many app architectures exist in software development and Flutter
- There is no single “only architecture.”
- Different companies and developers use different architectures depending on:
 - App size
 - Team size
 - Complexity
 - Scalability needs
 - State management choice

Beginner-Friendly Folder Structure

- For now, we use simple structure:
- lib/
 - |
 - |—— screens/
 - |—— widgets/
 - |—— models/
 - |—— viewmodels/
 - |—— constants/
 - |—— services/

Code Organization Principles

Principle 1 — Single Responsibility

- Each file should do ONE job
- Example:
 - Screen file → UI
 - Service file → APIs
 - Model file → Data structure

Prepared by Ehtisham Rasheed

Principle 2 – Reusability

- Avoid duplicate code
- If same button appears many times
- Create:
 - CustomButton()

Principle 3 — Naming Consistency

- Use clear names
- Good:
 - login_screen.dart
 - user_model.dart
 - api_service.dart
- Bad:
 - abc.dart
 - test.dart
 - data.dart

Prepared by Ehtisham Rasheed

Principle 4 – Small Files

- Avoid very large files
- If a screen becomes huge
- Split into smaller widgets

Prepared by Ehtisham Rasheed

Creating a Model

Model

Example JSON

```
{  
  "id": 1,  
  "name": "Ali"  
}
```

```
class UserModel {  
  final int id;  
  final String name;  
  
  UserModel({  
    required this.id,  
    required this.name,  
  });  
  
  factory UserModel.fromJson(Map<String, dynamic> json) {  
    return UserModel(  
      id: json['id'],  
      name: json['name'],  
    );  
  }  
}
```

Creating an API Service

```
import 'dart:convert';
import 'package:http/http.dart' as http;

class ApiService {
  Future fetchPosts() async {
    final response = await http.get(
      Uri.parse('https://jsonplaceholder.typicode.com/posts'),
    );

    return jsonDecode(response.body);
  }
}
```